16

Logic Programming Languages

The objectives of this chapter are to introduce the concepts of logic programming and logic programming languages, including a brief description of a subset of Prolog. We begin with an introduction to predicate calculus, which is the basis for logic programming languages. This is followed by a discussion of how predicate calculus can be used for automatic theorem-proving systems. Then, we present a general overview of logic programming. Next, a lengthy section introduces the basics of the Prolog programming language, including arithmetic, list processing, and a trace tool that can be used to help debug programs and also to illustrate how the Prolog system works. The final two sections describe some of the problems of Prolog as a logic language and some of the application areas in which Prolog has been used.

16.1 Introduction

Chapter 15 discusses the functional programming paradigm, which is significantly different from the software development methodologies used with the imperative languages. In this chapter, we describe another different programming methodology. In this case, the approach is to express programs in a form of symbolic logic and use a logical inferencing process to produce results. Logic programs are declarative rather than procedural, which means that only the specifications of the desired results are stated rather than detailed procedures for producing them. Programs in logic programming languages are collections of facts and rules. Such a program is used by asking it questions, which it attempts to answer by consulting the facts and rules. "Consulting" is perhaps misleading here, for the process is far more complex than that word connotes. This approach to problem solving may sound like it addresses only a very narrow category of problems, but it is more flexible than might be thought.

Programming that uses a form of symbolic logic as a programming language is often called **logic programming**, and languages based on symbolic logic are called **logic programming languages**, or **declarative languages**. We have chosen to describe the logic programming language Prolog, because it is the only widely used logic language.

The syntax of logic programming languages is remarkably different from that of the imperative and functional languages. The semantics of logic programs also bears little resemblance to that of imperative-language programs. These observations should lead the reader to some curiosity about the nature of logic programming and declarative languages.

16.2 A Brief Introduction to Predicate Calculus

Before we can discuss logic programming, we must briefly investigate its basis, which is formal logic. This is not our first contact with formal logic in this text; it was used extensively in the axiomatic semantics described in Chapter 3.

A **proposition** can be thought of as a logical statement that may or may not be true. It consists of objects and the relationships among objects. Formal logic was developed to provide a method for describing propositions, with the goal of allowing those formally stated propositions to be checked for validity.

Symbolic logic can be used for the three basic needs of formal logic: to express propositions, to express the relationships between propositions, and to describe how new propositions can be inferred from other propositions that are assumed to be true.

There is a close relationship between formal logic and mathematics. In fact, much of mathematics can be thought of in terms of logic. The fundamental axioms of number and set theory are the initial set of propositions, which are assumed to be true. Theorems are the additional propositions that can be inferred from the initial set.

The particular form of symbolic logic that is used for logic programming is called **first-order predicate calculus** (though it is a bit imprecise, we will usually refer to it as *predicate calculus*). In the following subsections, we present a brief look at predicate calculus. Our goal is to lay the groundwork for a discussion of logic programming and the logic programming language Prolog.

16.2.1 Propositions

The objects in logic programming propositions are represented by simple terms, which are either constants or variables. A constant is a symbol that represents an object. A variable is a symbol that can represent different objects at different times, although in a sense that is far closer to mathematics than the variables in an imperative programming language.

The simplest propositions, which are called **atomic propositions**, consist of compound terms. A **compound term** is one element of a mathematical relation, written in a form that has the appearance of mathematical function notation. Recall from Chapter 15 that a mathematical function is a mapping, which can be represented either as an expression or as a table or list of tuples. Compound terms are elements of the tabular definition of a function.

A compound term is composed of two parts: a **functor**, which is the function symbol that names the relation, and an ordered list of parameters, which together represent an element of the relation. A compound term with a single parameter is a 1-tuple; one with two parameters is a 2-tuple, and so forth. For example, we might have the two propositions

```
man (jake)
like (bob, steak)
```

which state that {jake} is a 1-tuple in the relation named man, and that {bob, steak} is a 2-tuple in the relation named like. If we added the proposition

```
man (fred)
```

to the two previous propositions, then the relation man would have two distinct elements, {jake} and {fred}. All of the simple terms in these propositions—man, jake, like, bob, and steak—are constants. Note that these propositions have no intrinsic semantics. They mean whatever we want them to mean. For example, the second example may mean that bob likes steak, or that steak likes bob, or that bob is in some way similar to a steak.

Propositions can be stated in two modes: one in which the proposition is defined to be true, and one in which the truth of the proposition is something that is to be determined. In other words, propositions either can be facts or queries. The example propositions above could be either.

Compound propositions have two or more atomic propositions, which are connected by logical connectors, or operators, in the same way compound logic expressions are constructed in imperative languages. The names, symbols, and meanings of the predicate calculus logical connectors are as follows:

Name	*Symbol*	*Example*	*Meaning*
negation	$\neg$	$\neg$ a	not *a*
conjunction	$\cap$	*a* $\cap$ *b*	*a* and *b*
disjunction	$\cup$	*a* $\cup$ *b*	*a* or *b*
equivalence	$\equiv$	*a* $\equiv$ *b*	*a* is equivalent to *b*
implication	$\supset$	*a* $\supset$ *b*	*a* implies *b*
	$\subset$	*a* $\subset$ *b*	*b* implies *a*

The following are examples of compound propositions:

a $\cap$ b $\supset$ c
a $\cap$ $\neg$ b $\supset$ d

The $\neg$ operator has the highest precedence. The operators $\cap$, $\cup$, and $\equiv$ all have higher precedence than $\supset$ and $\subset$. So, the second example above is equivalent to

(a $\cap$ ($\neg$ b)) $\supset$ d

Variables can appear in propositions but only when introduced by special symbols called *quantifiers*. Predicate calculus includes two quantifiers, as described below, where *X* is a variable and *P* is a proposition:

Name	*Example*	*Meaning*
universal	$\forall X.P$	For all *X*, *P* is true.
existential	$\exists X.P$	There exists a value of *X* such that *P* is true.

The period between X and P simply separates the variable from the proposition. For example, consider the following:

$\forall X.$ (woman $(X) \supset$ human (X))
$\exists X.$ (mother (mary, X) $\cap$ male (X))

The first of these propositions means that for any value of X, if X is a woman, then X is a human. The second means that there exists a value of X such that mary is the mother of X and X is a male; in other words, mary has a son. The scope of the universal and existential quantifiers is the atomic propositions to which they are attached. This scope can be extended using parentheses, as in the two compound propositions just described. So, the universal and existential quantifiers have higher precedence than any of the operators.

16.2.2 Clausal Form

We are discussing predicate calculus because it is the basis for logic programming languages. As with other languages, logic languages are best in their simplest form, meaning that redundancy should be minimized.

One problem with predicate calculus as we have described it thus far is that there are too many different ways of stating propositions that have the same meaning; that is, there is a great deal of redundancy. This is not such a problem for logicians, but if predicate calculus is to be used in an automated (computerized) system, it is a serious problem. To simplify matters, a standard form for propositions is desirable. Clausal form, which is a relatively simple form of propositions, is one such standard form. All propositions can be expressed in clausal form. A proposition in clausal form has the following general syntax:

$$B_1 \cup B_2 \cup \ldots \cup B_n \subset A_1 \cap A_2 \cap \ldots \cap A_m$$

in which the A's and B's are terms. The meaning of this clausal form proposition is as follows: If all of the A's are true, then at least one B is true. The primary characteristics of clausal form propositions are the following: Existential quantifiers are not required; universal quantifiers are implicit in the use of variables in the atomic propositions; and no operators other than conjunction and disjunction are required. Also, conjunction and disjunction need appear only in the order shown in the general clausal form: disjunction on the left side and conjunction on the right side. All predicate calculus propositions can be algorithmically converted to clausal form. Nilsson (1971) gives proof that this can be done, as well as a simple conversion algorithm for doing it.

The right side of a clausal form proposition is called the **antecedent**. The left side is called the **consequent** because it is the consequence of the truth of the antecedent. As examples of clausal form propositions, consider the following:

likes (bob, trout) $\subset$ likes (bob, fish) $\cap$ fish (trout)

$$\text{father}(\text{louis, al}) \cup \text{father}(\text{louis, violet}) \subset \text{father}(\text{al, bob}) \cap \text{mother}(\text{violet, bob}) \cap \text{grandfather}(\text{louis, bob})$$

The English version of the first of these states that if bob likes fish and a trout is a fish, then bob likes trout. The second states that if al is bob's father and violet is bob's mother and louis is bob's grandfather, then louis is either al's father or violet's father.

16.3 Predicate Calculus and Proving Theorems

Predicate calculus provides a method of expressing collections of propositions. One use of collections of propositions is to determine whether any interesting or useful facts can be inferred from them. This is exactly analogous to the work of mathematicians, who strive to discover new theorems that can be inferred from known axioms and theorems.

The early days of computer science (the 1950s and early 1960s) saw a great deal of interest in automating the theorem-proving process. One of the most significant breakthroughs in automatic theorem proving was the discovery of the resolution principle by Alan Robinson (1965) at Syracuse University.

Resolution is an inference rule that allows inferred propositions to be computed from given propositions, thus providing a method with potential application to automatic theorem proving. Resolution was devised to be applied to propositions in clausal form. The concept of resolution is the following: Suppose there are two propositions with the forms

$$P_1 \subset P_2$$
$$Q_1 \subset Q_2$$

Their meaning is that P_2 implies P_1 and Q_2 implies Q_1. Furthermore, suppose that P_1 is identical to Q_2, so that we could rename P_1 and Q_2 as T. Then, we could rewrite the two propositions as

$$T \subset P_2$$
$$Q_1 \subset T$$

Now, because P_2 implies T and T implies Q_1, it is logically obvious that P_2 implies Q_1, which we could write as

$$Q_1 \subset P_2$$

The process of inferring this proposition from the original two propositions is resolution.

As another example, consider the two propositions:

$$\text{older}(\text{joanne, jake}) \subset \text{mother}(\text{joanne, jake})$$
$$\text{wiser}(\text{joanne, jake}) \subset \text{older}(\text{joanne, jake})$$

From these propositions, the following proposition can be constructed using resolution:

wiser (joanne, jake) $\subset$ mother (joanne, jake)

The mechanics of this resolution construction are simple: The terms of the left sides of the two clausal propositions are OR'd together to make the left side of the new proposition. Then the right sides of the two clausal propositions are AND'd together to get the right side of the new proposition. Next, any term that appears on both sides of the new proposition is removed from both sides. The process is exactly the same when the propositions have multiple terms on either or both sides. The left side of the new inferred proposition initially contains all of the terms of the left sides of the two given propositions. The new right side is similarly constructed. Then the term that appears on both sides of the new proposition is removed. For example, if we have

father (bob, jake) $\cup$ mother (bob, jake) $\subset$ parent (bob, jake)
grandfather (bob, fred) $\subset$ father (bob, jake) $\cap$ father (jake, fred)

resolution says that

mother (bob, jake) $\cup$ grandfather (bob, fred) $\subset$
parent (bob, jake) $\cap$ father (jake, fred)

which has all but one of the atomic propositions of both of the original propositions. The one atomic proposition that allowed the operation father (bob, jake) in the left side of the first and in the right side of the second is left out. In English, we would say

if: bob is the parent of jake implies that bob is either the father or mother of jake
and: bob is the father of jake and jake is the father of fred implies that bob is the grandfather of fred
then: *if* bob is the parent of jake and jake is the father of fred *then:* either bob is jake's mother or bob is fred's grandfather

Resolution is actually more complex than these simple examples illustrate. In particular, the presence of variables in propositions requires resolution to find values for those variables that allow the matching process to succeed. This process of determining useful values for variables is called **unification**. The temporary assigning of values to variables to allow unification is called **instantiation**.

It is common for the resolution process to instantiate a variable with a value, fail to complete the required matching, and then be required to backtrack and instantiate the variable with a different value. We will discuss unification and backtracking more extensively in the context of Prolog.

A critically important property of resolution is its ability to detect any inconsistency in a given set of propositions. This is based on the formal property of resolution called **refutation complete**. What this means is that given a set of inconsistent propositions, resolution can prove them to be inconsistent. This allows resolution to be used to prove theorems, which can be done as

follows: We can envision a theorem proof in terms of predicate calculus as a given set of pertinent propositions, with the negation of the theorem itself stated as a new proposition. The theorem is negated so that resolution can be used to prove the theorem by finding an inconsistency. This is proof by contradiction, a frequently used approach to proving theorems in mathematics. Typically, the original propositions are called the **hypotheses**, and the negation of the theorem is called the **goal**.

Theoretically, this process is valid and useful. The time required for resolution, however, can be a problem. Although resolution is a finite process when the set of propositions is finite, the time required to find an inconsistency in a large database of propositions may be huge.

Theorem proving is the basis for logic programming. Much of what is computed can be couched in the form of a list of given facts and relationships as hypotheses, and a goal to be inferred from the hypotheses, using resolution.

Resolution on a hypotheses and a goal that are general propositions, even if they are in clausal form, is often not practical. Although it may be possible to prove a theorem using clausal form propositions, it may not happen in a reasonable amount of time. One way to simplify the resolution process is to restrict the form of the propositions. One useful restriction is to require the propositions to be Horn clauses. **Horn clauses** only can be in one of two forms: They have either a single atomic proposition on the left side or an empty left side.[1] The left side of a clausal form proposition is sometimes called the head, and Horn clauses with left sides are called headed Horn clauses. Headed Horn clauses are used to state relationships, such as

$$\text{likes}(\text{bob}, \text{trout}) \subset \text{likes}(\text{bob}, \text{fish}) \cap \text{fish}(\text{trout})$$

Horn clauses with empty left sides, which are often used to state facts, are called *headless Horn clauses*. For example,

$$\text{father}(\text{bob}, \text{jake})$$

Most, but not all, propositions can be stated as Horn clauses. The restriction to Horn clauses makes resolution a practical process for proving theorems.

16.4 An Overview of Logic Programming

Languages used for logic programming are called *declarative languages*, because programs written in them consist of declarations rather than assignments and control flow statements. These declarations are actually statements, or propositions, in symbolic logic.

One of the essential characteristics of logic programming languages is their semantics, which is called **declarative semantics**. The basic concept of this semantics is that there is a simple way to determine the meaning of each statement, and it does not depend on how the statement might be used to solve a

1. Horn clauses are named after Alfred Horn (1951), who studied clauses in this form.

problem. Declarative semantics is considerably simpler than the semantics of the imperative languages. For example, the meaning of a given proposition in a logic programming language can be concisely determined from the statement itself. In an imperative language, the semantics of a simple assignment statement requires examination of local declarations, knowledge of the scoping rules of the language, and possibly even examination of programs in other files just to determine the types of the variables in the assignment statement. Then, assuming the expression of the assignment contains variables, the execution of the program prior to the assignment statement must be traced to determine the values of those variables. The resulting action of the statement, then, depends on its run-time context. Comparing this semantics with that of a proposition in a logic language, with no need to consider textual context or execution sequences, it is clear that declarative semantics is far simpler than the semantics of imperative languages. Thus, declarative semantics is often stated as one of the advantages that declarative languages have over imperative languages (Hogger, 1984, pp. 240–241).

Programming in both imperative and functional languages is primarily procedural, which means that the programmer knows *what* is to be accomplished by a program and instructs the computer on exactly *how* the computation is to be done. In other words, the computer is treated as a simple device that obeys orders. Everything that is computed must have every detail of that computation spelled out. Some believe that this is the essence of the difficulty of programming using imperative and functional languages.

Programming in a logic programming language is nonprocedural. Programs in such languages do not state exactly *how* a result is to be computed but rather describe the form of the result. The difference is that we assume the computer system can somehow determine *how* the result is to be computed. What is needed to provide this capability for logic programming languages is a concise means of supplying the computer with both the relevant information and a method of inference for computing desired results. Predicate calculus supplies the basic form of communication to the computer, and resolution provides the inference technique.

Sorting is commonly used to illustrate the difference between procedural and nonprocedural systems. In a language like Java, sorting is done by explaining in a Java program all of the details of some sorting algorithm to a computer that has a Java compiler. The computer, after translating the Java program into machine code or some interpretive intermediate code, follows the instructions and produces the sorted list.

In a nonprocedural language, it is necessary only to describe the characteristics of the sorted list: It is some permutation of the given list such that for each pair of adjacent elements, a given relationship holds between the two elements. To state this formally, suppose the list to be sorted is in an array named list that has a subscript range $1 \ldots n$. The concept of sorting the elements of the given list, named old_list, and placing them in a separate array, named new_list, can then be expressed as follows:

$$\text{sort}(\text{old\_list}, \text{new\_list}) \subset \text{permute}(\text{old\_list}, \text{new\_list}) \bigcap \text{sorted}(\text{new\_list})$$
$$\text{sorted}(\text{list}) \subset \forall j \text{ such that } 11 \leq j < n, \text{list}(j) \leq \text{list}(j + 1)$$

where permute is a predicate that returns true if its second parameter array is a permutation of its first parameter array.

From this description, the nonprocedural language system could produce the sorted list. That makes nonprocedural programming sound like the mere production of concise software requirements specifications, which is a fair assessment. Unfortunately, however, it is not that simple. Logic programs that use only resolution face serious problems of execution efficiency. In our example of sorting, if the list is long, the number of permutations is huge, and they must be generated and tested, one by one, until the one that is in order is found—a very lengthy process. Of course, one must consider the possibility that the best form of a logic language may not yet have been determined, and good methods of creating programs in logic programming languages for large problems have not yet been developed.

16.5 The Origins of Prolog

As was stated in Chapter 2, Alain Colmerauer and Phillippe Roussel at the University of Aix-Marseille, with some assistance from Robert Kowalski at the University of Edinburgh, developed the fundamental design of Prolog. Colmerauer and Roussel were interested in natural-language processing; Kowalski was interested in automated theorem proving. The collaboration between the University of Aix-Marseille and the University of Edinburgh continued until the mid-1970s. Since then, research on the development and use of the language has progressed independently at those two locations, resulting in, among other things, two syntactically different dialects of Prolog.

The development of Prolog and other research efforts in logic programming received limited attention outside of Edinburgh and Marseille until the announcement in 1981 that the Japanese government was launching a large research project called the Fifth Generation Computing Systems (FGCS; Fuchi, 1981; Moto-oka, 1981). One of the primary objectives of the project was to develop intelligent machines, and Prolog was chosen as the basis for this effort. The announcement of FGCS aroused a sudden strong interest in artificial intelligence and logic programming in researchers and the governments of the United States and several European countries.

After a decade of effort, the FGCS project was quietly dropped. Despite the great assumed potential of logic programming and Prolog, little of great significance had been discovered. This led to the decline in the interest in and use of Prolog, although it still has its proponents.

16.6 The Basic Elements of Prolog

There are now a number of different dialects of Prolog. These can be grouped into several categories: those that grew from the Marseille group, those that came from the Edinburgh group, and some dialects that have been developed

for microcomputers, such as micro-Prolog, which is described by Clark and McCabe (1984). The syntactic forms of these are somewhat different. Rather than attempt to describe the syntax of several dialects of Prolog or some hybrid of them, we have chosen one particular, widely available dialect, which is the one developed at Edinburgh. This form of the language is sometimes called **Edinburgh syntax**. Its first implementation was on a DEC System-10 (Warren et al., 1979). Prolog implementations are available for virtually all popular computer platforms, for example, from the Free Software Organization (`http://www.gnu.org`).

16.6.1 Terms

As with programs in other languages, Prolog programs consist of collections of statements. There are only a few kinds of statements in Prolog, but they can be complex. All Prolog statements, as well as Prolog data, are constructed from terms.

A Prolog **term** is a constant, a variable, or a structure. A constant is either an **atom** or an integer. Atoms are the symbolic values of Prolog and are similar to their counterparts in LISP. In particular, an atom is either a string of letters, digits, and underscores that begins with a lowercase letter or a string of any printable ASCII characters delimited by apostrophes.

A variable is any string of letters, digits, and underscores that begins with an uppercase letter or an underscore (_). Variables are not bound to types by declarations. The binding of a value, and thus a type, to a variable is called an **instantiation**. Instantiation occurs only in the resolution process. A variable that has not been assigned a value is called **uninstantiated**. Instantiations last only as long as it takes to satisfy one complete goal, which involves the proof or disproof of one proposition. Prolog variables are only distant relatives, in terms of both semantics and use, to the variables in the imperative languages.

The last kind of term is called a **structure**. Structures represent the atomic propositions of predicate calculus, and their general form is the same:

```
functor(parameter list)
```

The functor is any atom and is used to identify the structure. The parameter list can be any list of atoms, variables, or other structures. As discussed at length in the following subsection, structures are the means of specifying facts in Prolog. They can also be thought of as objects, in which case they allow facts to be stated in terms of several related atoms. In this sense, structures are relations, for they state relationships among terms. A structure is also a predicate when its context specifies it to be a query (question).

16.6.2 Fact Statements

Our discussion of Prolog statements begins with those statements used to construct the hypotheses, or database of assumed information—the statements from which new information can be inferred.

Prolog has two basic statement forms; these correspond to the headless and headed Horn clauses of predicate calculus. The simplest form of headless Horn clause in Prolog is a single structure, which is interpreted as an unconditional assertion, or fact. Logically, facts are simply propositions that are assumed to be true.

The following examples illustrate the kinds of facts one can have in a Prolog program. Notice that every Prolog statement is terminated by a period.

```
female(shelley).
male(bill).
female(mary).
male(jake).
father(bill, jake).
father(bill, shelley).
mother(mary, jake).
mother(mary, shelley).
```

These simple structures state certain facts about `jake`, `shelley`, `bill`, and `mary`. For example, the first states that `shelley` is a `female`. The last four connect their two parameters with a relationship that is named in the functor atom; for example, the fifth proposition might be interpreted to mean that `bill` is the `father` of `jake`. Note that these Prolog propositions, like those of predicate calculus, have no intrinsic semantics. They mean whatever the programmer wants them to mean. For example, the proposition

```
father(bill, jake).
```

could mean `bill` and `jake` have the same `father` or that `jake` is the `father` of `bill`. The most common and straightforward meaning, however, might be that `bill` is the `father` of `jake`.

16.6.3 Rule Statements

The other basic form of Prolog statement for constructing the database corresponds to a headed Horn clause. This form can be related to a known theorem in mathematics from which a conclusion can be drawn if the set of given conditions is satisfied. The right side is the antecedent, or *if* part, and the left side is the consequent, or *then* part. If the antecedent of a Prolog statement is true, then the consequent of the statement must also be true. Because they are Horn clauses, the consequent of a Prolog statement is a single term, while the antecedent can be either a single term or a conjunction.

Conjunctions contain multiple terms that are separated by logical AND operations. In Prolog, the AND operation is implied. The structures that specify atomic propositions in a conjunction are separated by commas, so one could consider the commas to be AND operators. As an example of a conjunction, consider the following:

```
female(shelley), child(shelley).
```

The general form of the Prolog headed Horn clause statement is

```
consequence :- antecedent_expression.
```

It is read as follows: "consequence can be concluded if the antecedent expression is true or can be made to be true by some instantiation of its variables." For example,

```
ancestor(mary, shelley) :- mother(mary, shelley).
```

states that if `mary` is the `mother` of `shelley`, then `mary` is an `ancestor` of `shelley`. Headed Horn clauses are called **rules**, because they state rules of implication between propositions.

As with clausal form propositions in predicate calculus, Prolog statements can use variables to generalize their meaning. Recall that variables in clausal form provide a kind of implied universal quantifier. The following demonstrates the use of variables in Prolog statements:

```
parent(X, Y) :- mother(X, Y).
parent(X, Y) :- father(X, Y).
grandparent(X, Z) :- parent(X, Y) , parent(Y, Z).
```

These statements give rules of implication among some variables, or universal objects. In this case, the universal objects are `X`, `Y`, and `Z`. The first rule states that if there are instantiations of `X` and `Y` such that `mother(X, Y)` is true, then for those same instantiations of `X` and `Y`, `parent(X, Y)` is true.

The = operator, which is an infix operator, succeeds if its two term operands are the same. For example, `X = Y`. The `not` operator, which is a unary operator, reverses its operand, in the sense that it succeeds if its operand fails. For example, `not(X = Y)` succeeds if `X` is not equal to `Y`.

16.6.4 Goal Statements

So far, we have described the Prolog statements for logical propositions, which are used to describe both known facts and rules that describe logical relationships among facts. These statements are the basis for the theorem-proving model. The theorem is in the form of a proposition that we want the system to either prove or disprove. In Prolog, these propositions are called **goals**, or **queries**. The syntactic form of Prolog goal statements is identical to that of headless Horn clauses. For example, we could have

```
man(fred).
```

to which the system will respond either `yes` or `no`. The answer `yes` means that the system has proved the goal was true under the given database of facts and

relationships. The answer `no` means that either the goal was determined to be false or the system was simply unable to prove it.

Conjunctive propositions and propositions with variables are also legal goals. When variables are present, the system not only asserts the validity of the goal but also identifies the instantiations of the variables that make the goal true. For example,

```
father(X, mike).
```

can be asked. The system will then attempt, through unification, to find an instantiation of `X` that results in a true value for the goal.

Because goal statements and some nongoal statements have the same form (headless Horn clauses), a Prolog implementation must have some means of distinguishing between the two. Interactive Prolog implementations do this by simply having two modes, indicated by different interactive prompts: one for entering fact and rule statements and one for entering goals. The user can change the mode at any time.

16.6.5 The Inferencing Process of Prolog

This section examines Prolog resolution. Efficient use of Prolog requires that the programmer know precisely what the Prolog system does with his or her program.

When a goal is a compound proposition, each of the facts (structures) is called a **subgoal**. To prove that a goal is true, the inferencing process must find a chain of inference rules and/or facts in the database that connect the goal to one or more facts in the database. For example, if Q is the goal, then either Q must be found as a fact in the database or the inferencing process must find a fact P_1 and a sequence of propositions $P_2, P_3 \ldots, P_n$ such that

P_2 :- P_1
P_3 :- P_2
. . .
Q :- P_n

Of course, the process can be and often is complicated by rules with compound right sides `and` rules with variables. The process of finding the Ps, when they exist, is basically a comparison, or matching, of terms with each other.

Because the process of proving a subgoal is done through a proposition-matching process, it is sometimes called **matching**. In some cases, proving a subgoal is called **satisfying** that subgoal.

Consider the following query:

```
man(bob).
```

This goal statement is the simplest kind. It is relatively easy for resolution to determine whether it is true or false: The pattern of this goal is compared with the facts and rules in the database. If the database includes the fact

```
man(bob).
```

the proof is trivial. If, however, the database contains the following fact and inference rule,

```
father(bob).
man(X) :- father(X).
```

Prolog would be required to find these two statements and use them to infer the truth of the goal. This would necessitate unification to instantiate `X` temporarily to `bob`.

Now consider the goal

```
man(X).
```

In this case, Prolog must match the goal against the propositions in the database. The first proposition that it finds that has the form of the goal, with any object as its parameter, will cause `X` to be instantiated with that object's value. `X` is then displayed as the result. If there is no proposition having the form of the goal, the system indicates, by saying `no`, that the goal cannot be satisfied.

There are two opposite approaches to attempting to match a given goal to a fact in the database. The system can begin with the facts and rules of the database and attempt to find a sequence of matches that lead to the goal. This approach is called **bottom-up resolution**, or **forward chaining**. The alternative is to begin with the goal and attempt to find a sequence of matching propositions that lead to some set of original facts in the database. This approach is called **top-down resolution**, or **backward chaining**. In general, backward chaining works well when there is a reasonably small set of candidate answers. The forward chaining approach is better when the number of possibly correct answers is large; in this situation, backward chaining would require a very large number of matches to get to an answer. Prolog implementations use backward chaining for resolution, presumably because its designers believed backward chaining was more suitable for a larger class of problems than forward chaining.

The following example illustrates the difference between forward and backward chaining. Consider the query:

```
man(bob).
```

Assume the database contains

```
father(bob).
man(X) :- father(X).
```

Forward chaining would search for and find the first proposition. The goal is then inferred by matching the first proposition with the right side of the second rule (`father(X)`) through instantiation of `X` to `bob` and then matching the left side of the second proposition to the goal. Backward chaining would first

match the goal with the left side of the second proposition (`man(X)`) through the instantiation of `X` to `bob`. As its last step, it would match the right side of the second proposition (now `father(bob)`) with the first proposition.

The next design question arises whenever the goal has more than one structure, as in our example. The question then is whether the solution search is done depth first or breadth first. A **depth-first** search finds a complete sequence of propositions—a proof—for the first subgoal before working on the others. A **breadth-first** search works on all subgoals of a given goal in parallel. Prolog's designers chose the depth-first approach primarily because it can be done with fewer computer resources. The breadth-first approach is a parallel search that can require a large amount of memory.

The last feature of Prolog's resolution mechanism that must be discussed is backtracking. When a goal with multiple subgoals is being processed and the system fails to show the truth of one of the subgoals, the system abandons the subgoal it cannot prove. It then reconsiders the previous subgoal, if there is one, and attempts to find an alternative solution to it. This backing up in the goal to the reconsideration of a previously proven subgoal is called **backtracking**. A new solution is found by beginning the search where the previous search for that subgoal stopped. Multiple solutions to a subgoal result from different instantiations of its variables. Backtracking can require a great deal of time and space because it may have to find all possible proofs to every subgoal. These subgoal proofs may not be organized to minimize the time required to find the one that will result in the final complete proof, which exacerbates the problem.

To solidify your understanding of backtracking, consider the following example. Assume that there is a set of facts and rules in the database and that Prolog has been presented with the following compound goal:

```
male(X), parent(X, shelley).
```

This goal asks whether there is an instantiation of `X` such that `X` is a `male` and `X` is a `parent` of `shelley`. As its first step, Prolog finds the first fact in the database with `male` as its functor. It then instantiates `X` to the parameter of the found fact, say `mike`. Then, it attempts to prove that `parent(mike, shelley)` is true. If it fails, it backtracks to the first subgoal, `male(X)`, and attempts to resatisfy it with some alternative instantiation of `X`. The resolution process may have to find every `male` in the database before it finds the one that is a `parent` of `shelley`. It definitely must find all `male`s to prove that the goal cannot be satisfied. Note that our example goal might be processed more efficiently if the order of the two subgoals were reversed. Then, only after resolution had found a `parent` of `shelley` would it try to match that person with the `male` subgoal. This is more efficient if `shelley` has fewer parents than there are `male`s in the database, which seems like a reasonable assumption. Section 16.7.1 discusses a method of limiting the backtracking done by a Prolog system.

Database searches in Prolog always proceed in the direction of first to last.

The following two subsections describe Prolog examples that further illustrate the resolution process.

16.6.6 Simple Arithmetic

Prolog supports integer variables and integer arithmetic. Originally, the arithmetic operators were functors, so that the sum of `7` and the variable `X` was formed with

```
+(7, X)
```

Prolog now allows a more abbreviated syntax for arithmetic with the **is** operator. This operator takes an arithmetic expression as its right operand and a variable as its left operand. All variables in the expression must already be instantiated, but the left-side variable cannot be previously instantiated. For example, in

```
A is B / 17 + C.
```

if `B` and `C` are instantiated but `A` is not, then this clause will cause `A` to be instantiated with the value of the expression. When this happens, the clause is satisfied. If either `B` or `C` is not instantiated or `A` is instantiated, the clause is not satisfied and no instantiation of `A` can take place. The semantics of an **is** proposition is considerably different from that of an assignment statement in an imperative language. This difference can lead to an interesting scenario. Because the **is** operator makes the clause in which it appears look like an assignment statement, a beginning Prolog programmer may be tempted to write a statement such as

```
Sum is Sum + Number.
```

which is never useful, or even legal, in Prolog. If `Sum` is not instantiated, the reference to it in the right side is undefined and the clause fails. If `Sum` is already instantiated, the clause fails, because the left operand cannot have a current instantiation when **is** is evaluated. In either case, the instantiation of `Sum` to the new value will not take place. (If the value of `Sum + Number` is required, it can be bound to some new name.)

Prolog does not have assignment statements in the same sense as imperative languages. They are simply not needed in most of the programming for which Prolog was designed. The usefulness of assignment statements in imperative languages often depends on the capability of the programmer to control the execution control flow of the code in which the assignment statement is embedded. Because this type of control is not always possible in Prolog, such statements are far less useful.

As a simple example of the use of numeric computation in Prolog, consider the following problem: Suppose we know the average speeds of several

automobiles on a particular racetrack and the amount of time they are on the track. This basic information can be coded as facts, and the relationship between speed, time, and distance can be written as a rule, as in the following:

```
speed(ford, 100).
speed(chevy, 105).
speed(dodge, 95).
speed(volvo, 80).
time(ford, 20).
time(chevy, 21).
time(dodge, 24).
time(volvo, 24).
distance(X, Y) :- speed(X, Speed),
                  time(X, Time),
                  Y is Speed * Time.
```

Now, queries can request the distance traveled by a particular car. For example, the query

```
distance(chevy, Chevy_Distance).
```

instantiates `Chevy_Distance` with the value 2205. The first two clauses in the right side of the distance computation statement instantiate the variables `Speed` and `Time` with the corresponding values of the given automobile functor. After satisfying the goal, Prolog also displays the name `Chevy_Distance` and its value.

At this point it is instructive to take an operational look at how a Prolog system produces results. Prolog has a built-in structure named `trace` that displays the instantiations of values to variables at each step during the attempt to satisfy a given goal. `trace` is used to understand and debug Prolog programs. To understand `trace`, it is best to introduce a different model of the execution of Prolog programs, called the **tracing model**.

The tracing model describes Prolog execution in terms of four events: (1) call, which occurs at the beginning of an attempt to satisfy a goal, (2) exit, which occurs when a goal has been satisfied, (3) redo, which occurs when backtrack causes an attempt to resatisfy a goal, and (4) fail, which occurs when a goal fails. Call and exit can be related directly to the execution model of a subprogram in an imperative language if processes like `distance` are thought of as subprograms. The other two events are unique to logic programming systems. In the following trace example, a trace of the computation of the value for `Chevy_Distance`, the goal requires no redo or fail events:

```
trace.
distance(chevy, Chevy_Distance).

(1) 1 Call: distance(chevy, _0)?
(2) 2 Call: speed(chevy, _5)?
```

```
(2) 2 Exit: speed(chevy, 105)
(3) 2 Call: time(chevy, _6)?
(3) 2 Exit: time(chevy, 21)
(4) 2 Call: _0 is 105*21?
(4) 2 Exit: 2205 is 105*21
(1) 1 Exit: distance(chevy, 2205)

Chevy_Distance = 2205
```

Symbols in the trace that begin with the underscore character (_) are internal variables used to store instantiated values. The first column of the trace indicates the subgoal whose match is currently being attempted. For example, in the example trace, the first line with the indication `(3)` is an attempt to instantiate the temporary variable `_6` with a `time` value for `chevy`, where `time` is the second term in the right side of the statement that describes the computation of `distance`. The second column indicates the call depth of the matching process. The third column indicates the current action.

To illustrate backtracking, consider the following example database and traced compound goal:

```
likes(jake, chocolate).
likes(jake, apricots).
likes(darcie, licorice).
likes(darcie, apricots).

trace.
likes(jake, X), likes(darcie, X).

(1) 1 Call: likes(jake, _0)?
(1) 1 Exit: likes(jake, chocolate)
(2) 1 Call: likes(darcie, chocolate)?
(2) 1 Fail: likes(darcie, chocolate)
(1) 1 Redo: likes(jake, _0)?
(1) 1 Exit: likes(jake, apricots)
(3) 1 Call: likes(darcie, apricots)?
(3) 1 Exit: likes(darcie, apricots)

X = apricots
```

One can think about Prolog computations graphically as follows: Consider each goal as a box with four ports—call, fail, exit, and redo. Control enters a goal in the forward direction through its call port. Control can also enter a goal from the reverse direction through its redo port. Control can also leave a goal in two ways: If the goal succeeded, control leaves through the exit port; if the goal failed, control leaves through the fail port. A model of the example is shown in Figure 16.1. In this example, control flows through each subgoal

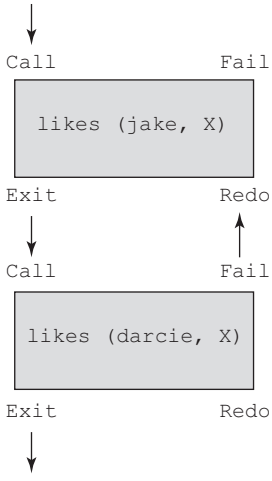

Figure 16.1

Control flow model for the goal `likes (jake, X), likes (darcie, X)`

twice. The second subgoal fails the first time, which forces a return through redo to the first subgoal.

16.6.7 List Structures

So far, the only Prolog data structure we have discussed is the atomic proposition, which looks more like a function call than a data structure. Atomic propositions, which are also called structures, are actually a form of records. The other basic data structure supported is the list. Lists are sequences of any number of elements, where the elements can be atoms, atomic propositions, or any other terms, including other lists.

Prolog uses the syntax of ML and Haskell to specify lists. The list elements are separated by commas, and the entire list is delimited by square brackets, as in

```
[apple, prune, grape, kumquat]
```

The notation `[]` is used to denote the empty list. Instead of having explicit functions for constructing and dismantling lists, Prolog simply uses a special notation. `[X | Y]` denotes a list with head `X` and tail `Y`, where head and tail correspond to `CAR` and `CDR` in LISP. This is similar to the notation used in ML and Haskell.

A list can be created with a simple structure, as in

```
new_list([apple, prune, grape, kumquat]).
```

which states that the constant list `[apple, prune, grape, kumquat]` is a new element of the relation named `new_list` (a name we just made up). This statement does not bind the list to a variable named `new_list`; rather, it does the kind of thing that the proposition

```
male(jake)
```

does. That is, it states that `[apple, prune, grape, kumquat]` is a new element of `new_list`. Therefore, we could have a second proposition with a list argument, such as

```
new_list([apricot, peach, pear])
```

In query mode, one of the elements of `new_list` can be dismantled into head and tail with

```
new_list([New_List_Head | New_List_Tail]).
```

If `new_list` has been set to have the two elements as shown, this statement instantiates `New_List_Head` with the head of the first list element (in this case, `apple`) and `New_List_Tail` with the tail of the list (or `[prune, grape, kumquat]`). If this were part of a compound goal and backtracking forced a new evaluation of it, `New_List_Head` and `New_List_Tail` would be reinstantiated to `apricot` and `[peach, pear]`, respectively, because `[apricot, peach, pear]` is the next element of `new_list`.

The `|` operator used to dismantle lists can also be used to create lists from given instantiated head and tail components, as in

```
[Element_1 | List_2]
```

If `Element_1` has been instantiated with `pickle` and `List_2` has been instantiated with `[peanut, prune, popcorn]`, the sample notation will create, for this one reference, the list `[pickle, peanut, prune, popcorn]`.

As stated previously, the list notation that includes the `|` symbol is universal: It can specify either a list construction or a list dismantling. Note further that the following are equivalent:

```
[apricot, peach, pear | []]
[apricot, peach | [pear]]
[apricot | [peach, pear]]
```

With lists, certain basic operations are often required, such as those found in LISP, ML, and Haskell. As an example of such operations in Prolog, we examine a definition of `append`, which is related to such a function in LISP. In this example, the differences and similarities between functional and declarative languages can be seen. We need not specify how Prolog is to construct a new list from the given lists; rather, we need specify only the characteristics of the new list in terms of the given lists.

In appearance, the Prolog definition of `append` is very similar to the ML version that appears in Chapter 15, and a kind of recursion in resolution is used in a similar way to produce the new list. In the case of Prolog, the recursion is caused and controlled by the resolution process. As with ML and Haskell,

a pattern-matching process is used to choose, based on the actual parameter, between two different definitions of the append process.

The first two parameters to the `append` operation in the following code are the two lists to be appended, and the third parameter is the resulting list:

```
append([], List, List).
append([Head | List_1], List_2, [Head | List_3]) :-
            append(List_1, List_2, List_3).
```

The first proposition specifies that when the empty list is appended to any other list, that other list is the result. This statement corresponds to the recursion-terminating step of the ML `append` function. Note that the terminating proposition is placed before the recursion proposition. This is done because we know that Prolog will match the two propositions in order, starting with the first (because of its use of the depth-first order).

The second proposition specifies several characteristics of the new list. It corresponds to the recursion step in the ML function. The left-side predicate states that the first element of the new list is the same as the first element of the first given list, because they are both named `Head`. Whenever `Head` is instantiated to a value, all occurrences of `Head` in the goal are, in effect, simultaneously instantiated to that value. The right side of the second statement specifies that the tail of the first given list `(List_1)` has the second given list `(List_2)` appended to it to form the tail `(List_3)` of the resulting list.

One way to read the second statement of `append` is as follows: Appending the list `[Head | List_1]` to any list `List_2` produces the list `[Head | List_3]`, but only if the list `List_3` is formed by appending `List_1` to `List_2`. In LISP, this would be

```
(CONS (CAR FIRST) (APPEND (CDR FIRST) SECOND))
```

In both the Prolog and LISP versions, the resulting list is not constructed until the recursion produces the terminating condition; in this case, the first list must become empty. Then, the resulting list is built using the `append` function itself; the elements taken from the first list are added, in reverse order, to the second list. The reversing is done by the unraveling of the recursion.

One fundamental difference between Prolog's `append` and those of LISP and ML is that Prolog's `append` is a predicate—it does not return a list, it returns `yes` or `no`. The new list is the value of its third parameter.

To illustrate how the `append` process progresses, consider the following traced example:

```
trace.
append([bob, jo], [jake, darcie], Family).

(1) 1 Call: append([bob, jo], [jake, darcie], _10)?
```

```
(2) 2 Call: append([jo], [jake, darcie], _18)?
(3) 3 Call: append([], [jake, darcie], _25)?
(3) 3 Exit: append([], [jake, darcie], [jake, darcie])
(2) 2 Exit: append([jo], [jake, darcie], [jo, jake,
                    darcie])
(1) 1 Exit: append([bob, jo], [jake, darcie],
                   [bob, jo, jake, darcie])
Family = [bob, jo, jake, darcie]
yes
```

The first two calls, which represent subgoals, have `List_1` nonempty, so they create the recursive calls from the right side of the second statement. The left side of the second statement effectively specifies the arguments for the recursive calls, or goals, thus dismantling the first list one element per step. When the first list becomes empty, in a call, or subgoal, the current instance of the right side of the second statement succeeds by matching the first statement. The effect of this is to return as the third parameter the value of the empty list appended to the second original parameter list. On successive exits, which represent successful matches, the elements that were removed from the first list are appended to the resulting list, `Family`. When the exit from the first goal is accomplished, the process is complete, and the resulting list is displayed.

Another difference between Prolog's `append` and those of LISP and ML is that Prolog's `append` is more flexible than that of those languages. For example, in Prolog we can use `append` to determine what two lists can be appended to get `[a, b, c]` with

```
append(X, Y, [a, b, c]).
```

This results in the following:

```
X = []
Y = [a, b, c]
```

If we type a semicolon at this output we get the alternative result:

```
X = [a]
Y = [b, c]
```

Continuing, we get the following:

```
X = [a, b]
Y = [c];
X = [a, b, c]
Y = []
```

The `append` predicate can also be used to create other list operations, such as the following, whose effect we invite the reader to determine. Note that `list_op_2` is meant to be used by providing a list as its first parameter and a variable as its second, and the result of `list_op_2` is the value to which the second parameter is instantiated.

```
list_op_2([], []).
list_op_2([Head | Tail], List) :-
list_op_2(Tail, Result), append(Result, [Head], List).
```

As you may have been able to determine, `list_op_2` causes the Prolog system to instantiate its second parameter with a list that has the elements of the list of the first parameter, but in reverse order. For example, `([apple, orange, grape], Q)` instantiates `Q` with the list `[grape, orange, apple]`.

Once again, although the LISP and Prolog languages are fundamentally different, similar operations can use similar approaches. In the case of the reverse operation, both the Prolog's `list_op_2` and LISP's `reverse` function include the recursion-terminating condition, along with the basic process of appending the reversal of the `CDR` or tail of the list to the `CAR` or head of the list to create the result list.

The following is a trace of this process, now named `reverse`:

```
trace.
reverse([a, b, c], Q).

(1) 1 Call: reverse([a, b, c], _6)?
(2) 2 Call: reverse([b, c], _65636)?
(3) 3 Call: reverse([c], _65646)?
(4) 4 Call: reverse([], _65656)?
(4) 4 Exit: reverse([], [])
(5) 4 Call: append([], [c], _65646)?
(5) 4 Exit: append([], [c], [c])
(3) 3 Exit: reverse([c], [c])
(6) 3 Call: append([c], [b], _65636)?
(7) 4 Call: append([], [b], _25)?
(7) 4 Exit: append([], [b], [b])
(6) 3 Exit: append([c], [b], [c, b])
(2) 2 Exit: reverse([b, c], [c, b])
(8) 2 Call: append([c, b], [a], _6)?
(9) 3 Call: append([b], [a], _32)?
(10) 4 Call: append([], [a], _39)?
(10) 4 Exit: append([], [a], [a])
(9) 3 Exit: append([b], [a], [b, a])
(8) 2 Exit: append([c, b], [a], [c, b, a])
(1) 1 Exit: reverse([a, b, c], [c, b, a])

Q = [c, b, a]
```

Suppose we need to be able to determine whether a given symbol is in a given list. A straightforward Prolog description of this is

```
member(Element, [Element | _]).
member(Element, [_ | List]) :- member(Element, List).
```

The underscore indicates an "anonymous" variable; it is used to mean that we do not care what instantiation it might get from unification. The first statement in the previous example succeeds if `Element` is the head of the list, either initially or after several recursions through the second statement. The second statement succeeds if `Element` is in the tail of the list. Consider the following traced examples:

```
trace.
member(a, [b, c, d]).
(1) 1 Call: member(a, [b, c, d])?
(2) 2 Call: member(a, [c, d])?
(3) 3 Call: member(a, [d])?
(4) 4 Call: member(a, [])?
(4) 4 Fail: member(a, [])
(3) 3 Fail: member(a, [d])
(2) 2 Fail: member(a, [c, d])
(1) 1 Fail: member(a, [b, c, d])
no

member(a, [b, a, c]).
(1) 1 Call: member(a, [b, a, c])?
(2) 2 Call: member(a, [a, c])?
(2) 2 Exit: member(a, [a, c])
(1) 1 Exit: member(a, [b, a, c])
yes
```

16.7 Deficiencies of Prolog

Although Prolog is a useful tool, it is neither a pure nor a perfect logic programming language. This section describes some of the problems with Prolog.

16.7.1 Resolution Order Control

Prolog, for reasons of efficiency, allows the user to control the ordering of pattern matching during resolution. In a pure logic programming environment, the order of attempted matches that take place during resolution is nondeterministic, and all matches could be attempted concurrently. However, because Prolog always matches in the same order, starting at the beginning of the database and at the left end of a given goal, the user can profoundly affect efficiency

by ordering the database statements to optimize a particular application. For example, if the user knows that certain rules are much more likely to succeed than the others during a particular "execution," then the program can be made more efficient by placing those rules at the beginning of the database.

In addition to allowing the user to control database and subgoal ordering, Prolog, in another concession to efficiency, allows some explicit control of backtracking. This is done with the cut operator, which is specified by an exclamation point (`!`). The cut operator is actually a goal, not an operator. As a goal, it always succeeds immediately, but it cannot be resatisfied through backtracking. Thus, a side effect of the cut is that subgoals to its left in a compound goal also cannot be resatisfied through backtracking. For example, in the goal

```
a, b, !, c, d.
```

if both `a` and `b` succeed but `c` fails, the whole goal fails. This goal would be used if it were known that whenever `c` fails, it is a waste of time to resatisfy `b` or `a`.

The purpose of the cut then is to allow the user to make programs more efficient by telling the system when it should not attempt to resatisfy subgoals that presumably could not result in a complete proof.

As an example of the use of the cut operator, consider the `member` rules from Section 16.6.7, which are:

```
member(Element, [Element | _]).
member(Element, [_ | List]) :- member(Element, List).
```

If the list argument to `member` represents a set, then it can be satisfied only once (sets contain no duplicate elements). Therefore, if `member` is used as a subgoal in a multiple subgoal goal statement, there can be a problem. The problem is that if `member` succeeds but the next subgoal fails, backtracking will attempt to resatisfy `member` by continuing a prior match. But because the list argument to `member` has only one copy of the element to begin with, `member` cannot possibly succeed again, which eventually causes the whole goal to fail, in spite of any additional attempts to resatisfy `member`. For example, consider the goal:

```
dem_candidate(X) :- member(X, democrats), tests(X).
```

This goal determines whether a given person is a democrat and is a good candidate to run for a particular position. The `tests` subgoal checks a variety of characteristics of the given democrat to determine the suitability of the person for the position. If the set of democrats has no duplicates, then we do not want to back up to the `member` subgoal if the `tests` subgoal fails, because `member` will search all of the other democrats but fail, because there are no duplicates. The second attempt of `member` subgoal will be a waste of computation time. The solution to this inefficiency is to add a right side to the first statement of the `member` definition, with the cut operator as the sole element, as in

```
member(Element, [Element | _]) :- !.
```

Backtracking will not attempt to resatisfy `member` but instead will cause the entire subgoal to fail.

Cut is particularly useful in a programming strategy in Prolog called **generate and test**. In programs that use the generate-and-test strategy, the goal consists of subgoals that generate potential solutions, which are then checked by later "test" subgoals. Rejected solutions require backtracking to "generator" subgoals, which generate new potential solutions. As an example of a generate-and-test program, consider the following, which appears in Clocksin and Mellish (2013):

```
divide(N1, N2, Result) :- is_integer(Result),
                          Product1 is Result * N2,
                          Product2 is (Result + 1) * N2,
                          Product1 =< N1, Product2 >
                            N1, !.
```

This program performs integer division, using addition and multiplication. Because most Prolog systems provide division as an operator, this program actually is not useful, other than to illustrate a simple generate-and-test program.

The predicate `is_integer` succeeds as long as its parameter can be instantiated to some nonnegative integer. If its argument is not instantiated, `is_integer` instantiates it to the value 0. If the argument is instantiated to an integer, `is_integer` instantiates it to the next larger integer value.

So, in `divide`, `is_integer` is the generator subgoal. It generates elements of the sequence 0, 1, 2, . . . , one each time it is satisfied. All of the other subgoals are the testing subgoals—they check to determine whether the value produced by `is_integer` is, in fact, the quotient of the first two parameters, `N1` and `N2`. The purpose of the cut as the last subgoal is simple: It prevents `divide` from ever trying to find an alternative solution once it has found *the* solution. Although `is_integer` can generate a huge number of candidates, only one is the solution, so the cut here prevents useless attempts to produce secondary solutions.

Use of the cut operator has been compared to the use of the goto in imperative languages (van Emden, 1980). Although it is sometimes needed, it is possible to abuse it. Indeed, it is sometimes used to make logic programs have a control flow that is inspired by imperative programming styles.

The ability to tamper with control flow in a Prolog program is a deficiency, because it is directly detrimental to one of the important advantages of logic programming—that programs do not specify how solutions are to be found. Rather, they simply specify what the solution should look like. This design makes programs easier to write and easier to read. They are not cluttered with the details of how the solutions are to be determined and, in particular, the precise order in which the computations are done to produce the solution. So, while logic programming requires no control flow directions, Prolog programs frequently use them, mostly for the sake of efficiency.

16.7.2 The Closed-World Assumption

The nature of Prolog's resolution sometimes creates misleading results. The only truths, as far as Prolog is concerned, are those that can be proved using its database. It has no knowledge of the world other than its database. When the system receives a query and the database does not have information to prove the query absolutely, the query is assumed to be false. Prolog can prove that a given goal is true, but it cannot prove that a given goal is false. It simply assumes that, because it cannot prove a goal true, the goal must be false. In essence, Prolog is a true/fail system, rather than a true/false system.

Actually, the closed-world assumption should not be at all foreign to you—our judicial system operates the same way. Suspects are innocent until proven guilty. They need not be proven innocent. If a trial cannot prove a person guilty, he or she is considered innocent.

The problem of the closed-world assumption is related to the negation problem, which is discussed in the following subsection.

16.7.3 The Negation Problem

Another problem with Prolog is its difficulty with negation. Consider the following database of two facts and a relationship:

```
parent(bill, jake).
parent(bill, shelley).
sibling(X, Y) :- (parent(M, X), parent(M, Y).
```

Now, suppose we typed the query

```
sibling(X, Y).
```

Prolog will respond with

```
X = jake
Y = jake
```

Thus, Prolog "thinks" `jake` is a `sibling` of himself. This happens because the system first instantiates `M` with `bill` and `X` with `jake` to make the first subgoal, `parent(M, X)`, true. It then starts at the beginning of the database again to match the second subgoal, `parent(M, Y)`, and arrives at the instantiations of `M` with `bill` and `Y` with `jake`. Because the two subgoals are satisfied independently, with both matchings starting at the database's beginning, the shown response appears. To avoid this result, `X` must be specified to be a `sibling` of `Y` only if they have the same `parents` *and* they are not the same. Unfortunately, stating that they are not equal is not straightforward in Prolog, as we will discuss. The most exacting method would require adding a fact for every pair of atoms, stating that they were not the same. This can, of course, cause the database to become very large, for there is often far more negative information than

positive information. For example, most people have 364 more unbirthdays than they have birthdays.

A simple alternative solution is to state in the goal that `X` must not be the same as `Y`, as in

```
sibling(X, Y) :- parent(M, X), parent(M, Y), not(X = Y).
```

In other situations, the solution is not so simple.

The Prolog `not` operator is satisfied in this case if resolution cannot satisfy the subgoal `X = Y`. Therefore, if the `not` succeeds, it does not necessarily mean that `X` is not equal to `Y`; rather, it means that resolution cannot prove from the database that `X` is the same as `Y`. Thus, the Prolog `not` operator is not equivalent to a logical NOT operator, in which NOT means that its operand is probably true. This nonequivalency can lead to a problem if we happen to have a goal of the form

```
not(not(some_goal)).
```

which would be equivalent to

```
some_goal.
```

if Prolog's `not` operator were a true logical NOT operator. In some cases, however, they are not the same. For example, consider again the `member` rules:

```
member(Element, [Element | _]) :- !.
member(Element, [_ | List]) :- member(Element, List).
```

To discover one of the elements of a given list, we could use the goal

```
member(X, [mary, fred, barb]).
```

which would cause `X` to be instantiated with `mary`, which would then be printed. But if we used

```
not(not(member(X, [mary, fred, barb]))).
```

the following sequence of events would take place: First, the inner goal would succeed, instantiating `X` to `mary`. Then, Prolog would attempt to satisfy the next goal:

```
not(member(X, [mary, fred, barb])).
```

This statement would fail because `member` succeeded. When this goal failed, `X` would be uninstantiated, because Prolog always uninstantiates all variables in all goals that fail. Next, Prolog would attempt to satisfy the outer `not` goal,

which would succeed, because its argument had failed. Finally, the result, which is X, would be printed. But X would not be currently instantiated, so the system would indicate that. Generally, uninstantiated variables are printed in the form of a string of digits preceded by an underscore. So, the fact that Prolog's `not` is not equivalent to a logical NOT can be, at the very least, misleading.

The fundamental reason why logical NOT cannot be an integral part of Prolog is the form of the Horn clause:

$$A \text{ :- } B_1 \cap B_2 \cap \ldots \cap B_n$$

If all the B propositions are true, it can be concluded that A is true. But regardless of the truth or falseness of any or all of the B's, it cannot be concluded that A is false. From positive logic, one can conclude only positive logic. Thus, the use of Horn clause form prevents any negative conclusions.

16.7.4 Intrinsic Limitations

A fundamental goal of logic programming, as stated in Section 16.4, is to provide nonprocedural programming; that is, a system by which programmers specify what a program is supposed to do but need not specify how that is to be accomplished. The example given there for sorting is rewritten here:

$$\text{sort}(\text{old\_list}, \text{new\_list}) \subset \text{permute}(\text{old\_list}, \text{new\_list}) \cap \text{sorted}(\text{new\_list})$$

$$\text{sorted}(\text{list}) \subset \forall j \text{ such that } 1 \leq j < n, \text{list}(j) \leq \text{list}(j+1)$$

It is straightforward to write this in Prolog. For example, the sorted subgoal can be expressed as

```
sorted ([]).
sorted ([x]).
sorted ([x, y | list]) :- x <= y, sorted ([y | list]).
```

The problem with this sort process is that it has no idea of how to sort, other than simply to enumerate all permutations of the given list until it happens to create the one that has the list in sorted order—a very slow process, indeed.

So far, no one has discovered a process by which the description of a sorted list can be transformed into some efficient algorithm for sorting. Resolution is capable of many interesting things, but certainly not this. Therefore, a Prolog program that sorts a list must specify the details of how that sorting can be done, as is the case in an imperative or functional language.

Do all of these problems mean that logic programming should be abandoned? Absolutely not! As it is, it is capable of dealing with many useful applications. Furthermore, it is based on an intriguing concept and is therefore interesting in and of itself. Finally, there is the possibility that new inferencing techniques will be developed that will allow a logic programming language system to efficiently deal with progressively larger classes of problems.

16.8 Applications of Logic Programming

In this section, we briefly describe a few of the larger classes of present and potential applications of logic programming in general and Prolog in particular.

16.8.1 Relational Database Management Systems

Relational database management systems (RDBMSs) store data in the form of tables. Queries on such databases are often stated in Structured Query Language (SQL). SQL is nonprocedural in the same sense that logic programming is nonprocedural. The user does not describe how to retrieve the answer; rather, he or she describes only the characteristics of the answer. The connection between logic programming and RDBMSs should be obvious. Simple tables of information can be described by Prolog structures, and relationships between tables can be conveniently and easily described by Prolog rules. The retrieval process is inherent in the resolution operation. The goal statements of Prolog provide the queries for the RDBMS. Logic programming is thus a natural match to the needs of implementing an RDBMS.

One of the advantages of using logic programming to implement an RDBMS is that only a single language is required. In a typical RDBMS, a database language includes statements for data definitions, data manipulation, and queries, all of which are embedded in a general-purpose programming language, such as COBOL. The general-purpose language is used for processing the data and input and output functions. All of these functions can be done in a logic programming language.

Another advantage of using logic programming to implement an RDBMS is that deductive capability is built in. Conventional RDBMSs cannot deduce anything from a database other than what is explicitly stored in them. They contain only facts, rather than facts *and* inference rules. The primary disadvantage of using logic programming for an RDBMS, compared with a conventional RDBMS, is that the logic programming implementation is slower. Logical inferences take longer than ordinary table look-up methods using imperative programming techniques.

16.8.2 Expert Systems

Expert systems are computer systems designed to emulate human expertise in some particular domain. They consist of a database of facts, an inferencing process, some heuristics about the domain, and some friendly human interface that makes the system appear much like an expert human consultant. In addition to their initial knowledge base, which is provided by a human expert, expert systems learn from the process of being used, so their databases must be capable of growing dynamically. Also, an expert system should include the capability of interrogating the user to get additional information when it determines that such information is needed.

One of the central problems for the designer of an expert system is dealing with the inevitable inconsistencies and incompleteness of the database. Logic programming appears to be well suited to deal with these problems. For example, default inference rules can help deal with the problem of incompleteness.

Prolog can and has been used to construct expert systems. It can easily fulfill the basic needs of expert systems, using resolution as the basis for query processing, using its ability to add facts and rules to provide the learning capability, and using its trace facility to inform the user of the "reasoning" behind a given result. Missing from Prolog is the automatic ability of the system to query the user for additional information when it is needed.

One of the most widely known uses of logic programming in expert systems is the expert system construction system known as APES, which is described in Sergot (1983) and Hammond (1983). The APES system includes a very flexible facility for gathering information from the user during expert system construction. It also includes a second interpreter for producing explanations to its answers to queries.

APES has been successfully used to produce several expert systems, including one for the rules of a government social benefits program and one for the British Nationality Act, which is the definitive source for rules of British citizenship.

16.8.3 Natural-Language Processing

Certain kinds of natural-language processing can be done with logic programming. In particular, natural-language interfaces to computer software systems, such as intelligent databases and other intelligent knowledge-based systems, can be conveniently done with logic programming. For describing language syntax, forms of logic programming have been found to be equivalent to context-free grammars. Proof procedures in logic programming systems have been found to be equivalent to certain parsing strategies. In fact, backward-chaining resolution can be used directly to parse sentences whose structures are described by context-free grammars. It has also been discovered that some kinds of semantics of natural languages can be made clear by modeling the languages with logic programming. In particular, research in logic-based semantics networks has shown that sets of sentences in natural languages can be expressed in clausal form (Deliyanni and Kowalski, 1979). Kowalski (1979) also discusses logic-based semantic networks.

SUMMARY

Symbolic logic provides the basis for logic programming and logic programming languages. The approach of logic programming is to use as a database a collection of facts and rules that state relationships between facts and to use an automatic inferencing process to check the validity of new propositions,

assuming the facts and rules of the database are true. This approach is the one developed for automatic theorem proving.

Prolog is the most widely used logic programming language. The origins of logic programming lie in Robinson's development of the resolution rule for logical inference. Prolog was developed primarily by Colmeraeur and Roussel at Marseille, with some help from Kowalski at Edinburgh.

Logic programs are nonprocedural, which means that the characteristics of the solution are given but the process of getting the solution is not.

Prolog statements are facts, rules, or goals. Most are made up of structures, which are atomic propositions, and logic operators, although arithmetic expressions are also allowed.

Resolution is the primary activity of a Prolog interpreter. This process, which uses backtracking extensively, involves mainly pattern matching among propositions. When variables are involved, they can be instantiated to values to provide matches. This instantiation process is called *unification*.

There are a number of problems with the current state of logic programming. For reasons of efficiency, and even to avoid infinite loops, programmers must sometimes state control flow information in their programs. Also, there are the problems of the closed-world assumption and negation.

Logic programming has been used in a number of different areas, primarily in relational database systems, expert systems, and natural-language processing.

BIBLIOGRAPHIC NOTES

The Prolog language is described in several books. Edinburgh's form of the language is covered in Clocksin and Mellish (2003). The microcomputer implementation is described in Clark and McCabe (1984).

Hogger (1991) is an excellent book on the general topic of logic programming. It is the source of the material in this chapter's section on logic programming applications.

REVIEW QUESTIONS

1. What are the three primary uses of symbolic logic in formal logic?
2. What are the two parts of a compound term?
3. What are the two modes in which a proposition can be stated?
4. What is the general form of a proposition in clausal form?
5. What are antecedents? Consequences?
6. Give general (not rigorous) definitions of *resolution* and *unification*.
7. What are the forms of Horn clauses?

8. What is the basic concept of declarative semantics?
9. What does it mean for a language to be nonprocedural?
10. What are the three forms of a Prolog term?
11. What is an uninstantiated variable?
12. What are the syntactic forms and usage of fact and rule statements in Prolog?
13. What is a conjunction?
14. Explain the two approaches to matching goals to facts in a database.
15. Explain the difference between a depth-first and a breadth-first search when discussing how multiple goals are satisfied.
16. Explain how backtracking works in Prolog.
17. Explain what is wrong with the Prolog statement `K is K + 1`.
18. What are the two ways a Prolog programmer can control the order of pattern matching during resolution?
19. Explain the generate-and-test programming strategy in Prolog.
20. Explain the closed-world assumption used by Prolog. Why is this a limitation?
21. Explain the negation problem with Prolog. Why is this a limitation?
22. Explain the connection between automatic theorem proving and Prolog's inferencing process.
23. Explain the difference between procedural and nonprocedural languages.
24. Explain why Prolog systems must do backtracking.
25. What is the relationship between resolution and unification in Prolog?

PROBLEM SET

1. Compare the concept of data typing in C# with that of Prolog.
2. Describe how a multiple-processor machine could be used to implement resolution. Could Prolog, as currently defined, use this method?
3. Write a Prolog description of your family tree (based only on facts), going back to your grandparents and including all descendants. Be sure to include all relationships.
4. Write a set of rules for family relationships, including all relationships from grandparents through two generations. Now add these to the facts of Problem 3, and eliminate as many of the facts as you can.

5. Write the following English conditional statements as Prolog-headed Horn clauses:
   a. If Fred is the father of Mike, then Fred is an ancestor of Mike.
   b. If Mike is the father of Joe and Mike is the father of Mary, then Mary is the sister of Joe.
   c. If Mike is the brother of Fred and Fred is the father of Mary, then Mike is the uncle of Mary.
6. Explain two ways in which the list-processing capabilities of Scheme and Prolog are similar.
7. In what way are the list-processing capabilities of Scheme and Prolog different?
8. Write a comparison of Prolog with ML, including two similarities and two differences.
9. From a book on Prolog, learn and write a description of an occur-check problem. Why does Prolog allow this problem to exist in its implementation?
10. Find a good source of information on Skolem normal form and write a brief but clear explanation of it.

PROGRAMMING EXERCISES

1. Using the structures `parent(X, Y)`, `male(X)`, and `female(X)`, write a structure that defines `mother(X, Y)`.
2. Using the structures `parent(X, Y)`, `male(X)`, and `female(X)`, write a structure that defines `sister(X, Y)`.
3. Write a Prolog program that finds the maximum of a list of numbers.
4. Write a Prolog program that succeeds if the intersection of two given list parameters is empty.
5. Write a Prolog program that returns a list containing the union of the elements of two given lists.
6. Write a Prolog program that returns the final element of a given list.
7. Write a Prolog program that implements quicksort.

Bibliography

ACM. (1979) "Part A: Preliminary Ada Reference Manual" and "Part B: Rationale for the Design of the Ada Programming Language." SIGPLAN Notices, Vol. 14, No. 6.

ACM. (1993a) "History of Programming Language Conference Proceedings." ACM SIGPLAN Notices, Vol. 28, No. 3, March.

ACM. (1993b) "High Performance FORTRAN Language Specification Part 1." FORTRAN Forum, Vol. 12, No. 4.

Aho, A. V., B. W. Kernighan, and P. J. Weinberger. (1988) The AWK Programming Language. Addison-Wesley, Reading, MA.

Aho, A. V., M. S. Lam, R. Sethi, and J. D. Ullman. (2006) Compilers: Principles, Techniques, and Tools, 2e. Addison-Wesley, Reading, MA.

Albahari, J. and B. Abrahari (2012) C# 5.0 in a Nutshell, O'Reilly Media, Sebastopol, CA.

Andrews, G. R., and F. B. Schneider. (1983) "Concepts and Notations for Concurrent Programming." ACM Computing Surveys, Vol. 15, No. 1, pp. 3–43.

ANSI. (1966) American National Standard Programming Language FORTRAN. American National Standards Institute, New York.

ANSI. (1976) American National Standard Programming Language PL/I. ANSI X3.53–1976. American National Standards Institute, New York.

ANSI. (1978a) American National Standard Programming Language FORTRAN. ANSI X3.9–1978. American National Standards Institute, New York.

ANSI. (1978b) American National Standard Programming Language Minimal BASIC. ANSI X3.60–1978. American National Standards Institute, New York.

ANSI. (1989) American National Standard Programming Language C. ANSI X3.159–1989. American National Standards Institute, New York.

ANSI. (1992) American National Standard Programming Language FORTRAN 90. ANSI X3. 198–1992. American National Standards Institute, New York.

Arden, B. W., B. A. Galler, and R. M. Graham. (1961) "MAD at Michigan." Datamation, Vol. 7, No. 12, pp. 27–28.

ARM. (1995) Ada Reference Manual. ISO/IEC/ANSI 8652:19. Intermetrics, Cambridge, MA.

Arnold, K., J. Gosling, and D. Holmes. (2006) The Java (TM) Programming Language, 4e. Addison-Wesley, Reading, MA.

Backus, J. (1954) "The IBM 701 Speedcoding System." J. ACM, Vol. 1, pp. 4–6.

Backus, J. (1959) "The Syntax and Semantics of the Proposed International Algebraic Language of the Zurich ACM-GAMM Conference." Proceedings International Conference on Information Processing. UNESCO, Paris, pp. 125–132.

Backus, J. (1978) "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs." Commun. ACM, Vol. 21, No. 8, pp. 613–641.

Backus, J., F. L. Bauer, J. Green, C. Katz, J. McCarthy, P. Naur, A. J. Perlis, H. Rutishauser, K. Samelson, B. Vauquois, J. H. Wegstein, A. van Wijngaarden, and M. Woodger. (1963) "Revised Report on the Algorithmic Language ALGOL 60." Commun. ACM, Vol. 6, No. 1, pp. 1–17.

Ben-Ari, M. (1982) Principles of Concurrent Programming. Prentice Hall, Englewood Cliffs, NJ.

Birtwistle, G. M., O.-J. Dahl, B. Myhrhaug, and K. Nygaard. (1973) Simula BEGIN. Van Nostrand Reinhold, New York.

Bodwin, J. M., L. Bradley, K. Kanda, D. Litle, and U. F. Pleban. (1982) "Experience with an Experimental Compiler Generator Based on Denotational Semantics." ACM SIGPLAN Notices, Vol. 17, No. 6, pp. 216–229.

Bohm, C., and G. Jacopini. (1966) "Flow Diagrams, Turing Machines, and Languages with Only Two Formation Rules." Commun. ACM, Vol. 9, No. 5, pp. 366–371.

Bolsky, M., and D. Korn. (1995) The New KornShell Command and Programming Language. Prentice Hall, Englewood Cliffs, NJ.

Booch, G. (1987) Software Engineering with Ada, 2e. Benjamin/Cummings, Redwood City, CA.

Brinch Hansen, P. (1973) Operating System Principles. Prentice Hall, Englewood Cliffs, NJ.

Brinch Hansen, P. (1975) "The Programming Language Concurrent-Pascal." IEEE Transactions on Software Engineering, Vol. 1, No. 2, pp. 199–207.

Brinch Hansen, P. (1977) The Architecture of Concurrent Programs. Prentice Hall, Englewood Cliffs, NJ.

Brinch Hansen, P. (1978) "Distributed Processes: A Concurrent Programming Concept." Commun. ACM, Vol. 21, No. 11, pp. 934–941.

Brown, J. A., S. Pakin, and R. P. Polivka. (1988) APL2 at a Glance. Prentice Hall, Englewood Cliffs, NJ.

Campione, M., K. Walrath, and A. Huml. (2001) The Java Tutorial, 3e. Addison-Wesley, Reading, MA.

Chambers, C., and D. Ungar. (1991) "Making Pure Object-Oriented Languages Practical." SIGPLAN Notices, Vol. 26, No. 1, pp. 1–15.

Chomsky, N. (1956) "Three Models for the Description of Language." IRE Transactions on Information Theory, Vol. 2, No. 3, pp. 113–124.

Chomsky, N. (1959) "On Certain Formal Properties of Grammars." Information and Control, Vol. 2, No. 2, pp. 137–167.

Christiansen, T., B. D. Foy, and L. Wall, with J. Orwant. (2013) Programming Perl, 4e. O'Reilly & Associates, Sebastopol, CA.

Church, A. (1941) Annals of Mathematics Studies. Calculi of Lambda Conversion, Vol. 6. Princeton University Press, Princeton, NJ. Reprinted by Klaus Reprint Corporation, New York, 1965.

Clark, K. L., and F. G. McCabe. (1984) Micro-PROLOG: Programming in Logic. Prentice Hall, Englewood Cliffs, NJ.

Clarke, L. A., J. C. Wileden, and A. L. Wolf. (1980) "Nesting in Ada Is for the Birds." ACM SIGPLAN Notices, Vol. 15, No. 11, pp. 139–145.

Cleaveland, J. C. (1986) An Introduction to Data Types. Addison-Wesley, Reading, MA.
Cleaveland, J. C., and R. C. Uzgalis. (1976) Grammars for Programming Languages: What Every Programmer Should Know About Grammar. American Elsevier, New York.
Clocksin, W. F., and C. S. Mellish. (2013) Programming in Prolog: Using the ISO Standard. Springer-Verlag, New York.
Cohen, J. (1981) "Garbage Collection of Linked Data Structures." ACM Computing Surveys, Vol. 13, No. 3, pp. 341–368.
Conway, R., and R. Constable. (1976) "PL/-CS—A Disciplined Subset of PL/I." Technical Report TR76/293. Department of Computer Science, Cornell University, Ithaca, NY.
Cornell University. (1977) PL/C User's Guide, Release 7.6. Department of Computer Science, Cornell University, Ithaca, NY.
Correa, N. (1992) "Empty Categories, Chain Binding, and Parsing." In Principle—Based Parsing, R. C. Berwick, S. P. Abney, and C. Tenny (eds.). Kluwer Academic Publishers, Boston, pp. 83–121.
Cousineau, G., M. Mauny, and K. Callaway. (1998) The Functional Approach to Programming. Cambridge University Press, Cambridge, UK.
Dahl, O.-J., E. W. Dijkstra, and C. A. R. Hoare. (1972) Structured Programming. Academic Press, New York.
Dahl, O.-J., and K. Nygaard. (1967) SIMULA 67 Common Base Proposal. Norwegian Computing Center Document, Oslo.
Deliyanni, A., and R. A. Kowalski. (1979) "Logic and Semantic Networks." Commun. ACM, Vol. 22, No. 3, pp. 184–192.
Department of Defense. (1960) COBOL, Initial Specifications for a Common Business Oriented Language. U.S. Department of Defense, Washington, D.C.
Department of Defense. (1961) COBOL—1961, Revised Specifications for a Common Business Oriented Language. U.S. Department of Defense, Washington, D.C.
Department of Defense. (1962) COBOL—1961 EXTENDED, Extended Specifications for a Common Business Oriented Language. U.S. Department of Defense, Washington, D.C.
Department of Defense. (1975a) Requirements for High Order Programming Languages, STRAWMAN. July. U.S. Department of Defense, Washington, D.C.
Department of Defense. (1975b) Requirements for High Order Programming Languages, WOODENMAN. August. U.S. Department of Defense, Washington, D.C.
Department of Defense. (1976) Requirements for High Order Programming Languages, TINMAN. June. U.S. Department of Defense, Washington, D.C.
Department of Defense. (1977) Requirements for High Order Programming Languages, IRONMAN. January. U.S. Department of Defense, Washington, D.C.
Department of Defense. (1978) Requirements for High Order Programming Languages, STEELMAN. June. U.S. Department of Defense, Washington, D.C.
Department of Defense. (1980) Requirements for High Order Programming Languages, STONEMAN. February. U.S. Department of Defense, Washington, D.C.
DeRemer, F. (1971) "Simple LR(k) Grammars." Commun. ACM, Vol. 14, No. 7, pp. 453–460.
DeRemer, F., and T. Pennello. (1982) "Efficient Computation of LALR(1) Look-Ahead Sets." ACM TOPLAS, Vol. 4, No. 4, pp. 615–649.

Deutsch, L. P., and D. G. Bobrow. (1976) "An Efficient Incremental Automatic Garbage Collector." Commun. ACM, Vol. 11, No. 3, pp. 522–526.
Dijkstra, E. W. (1968a) "Goto Statement Considered Harmful." Commun. ACM, Vol. 11, No. 3, pp. 147–149.
Dijkstra, E. W. (1968b) "Cooperating Sequential Processes." In Programming Languages, F. Genuys (ed.). Academic Press, New York, pp. 43–112.
Dijkstra, E. W. (1972) "The Humble Programmer." Commun. ACM, Vol. 15, No. 10, pp. 859–866.
Dijkstra, E. W. (1975) "Guarded Commands, Nondeterminacy, and Formal Derivation of Programs." Commun. ACM, Vol. 18, No. 8, pp. 453–457.
Dijkstra, E. W. (1976) A Discipline of Programming. Prentice Hall, Englewood Cliffs, NJ.
Dybvig, R. K. (2011) The Scheme Programming Language, 4e. MIT Press, Boston.
Ellis, M. A., and B. Stroustrup. (1990) The Annotated C++ Reference Manual. Addison-Wesley, Reading, MA.
Farber, D. J., R. E. Griswold, and I. P. Polonsky. (1964) "SNOBOL, a String Manipulation Language." J. ACM, Vol. 11, No. 1, pp. 21–30.
Farrow, R. (1982) "LINGUIST 86: Yet Another Translator Writing System Based on Attribute Grammars." ACM SIGPLAN Notices, Vol. 17, No. 6, pp. 160–171.
Fischer, C. N., G. F. Johnson, J. Mauney, A. Pal, and D. L. Stock. (1984) "The Poe Language-Based Editor Project." ACM SIGPLAN Notices, Vol. 19, No. 5, pp. 21–29.
Fischer, C. N., and R. J. LeBlanc. (1977) UW-Pascal Reference Manual. Madison Academic Computing Center, Madison, WI.
Fischer, C. N., and R. J. LeBlanc. (1980) "Implementation of Runtime Diagnostics in Pascal." IEEE Transactions on Software Engineering, Vol. SE-6, No. 4, pp. 313–319.
Fischer, C. N., and R. J. LeBlanc. (1991) Crafting a Compiler with C. Benjamin-Cummings, Menlo Park, CA.
Flanagan, D. (2011) JavaScript: The Definitive Guide, 6e. O'Reilly Media, Sebastopol, CA.
Flanagan, D., and Y. Matsumoto. (2008) The Ruby Programming Language, O'Reilly Media, Sebastopol, CA.
Floyd, R. W. (1967) "Assigning Meanings to Programs." Proceedings Symposium Applied Mathematics. Mathematical Aspects of Computer Science, J. T. Schwartz (ed.). American Mathematical Society, Providence, RI.
Frege, G. (1892) "Über Sinn und Bedeutung." Zeitschrift für Philosophie und Philosophisches Kritik, Vol. 100, pp. 25–50.
Friedl, J. E. F. (2006) Mastering Regular Expressions, 3e. O'Reilly Media, Sebastopol, CA.
Friedman, D. P., and D. S. Wise. (1979) "Reference Counting's Ability to Collect Cycles Is Not Insurmountable." Information Processing Letters, Vol. 8, No. 1, pp. 41–45.
Fuchi, K. (1981) "Aiming for Knowledge Information Processing Systems." Proceedings of the International Conference on Fifth Generation Computing Systems. Japan Information Processing Development Center, Tokyo. Republished (1982) by North-Holland Publishing, Amsterdam.
Gehani, N. (1983) Ada: An Advanced Introduction. Prentice Hall, Englewood Cliffs, NJ.

Gilman, L., and A. J. Rose. (1983) APL: An Interactive Approach, 3e. John Wiley, New York.
Goodenough, J. B. (1975) "Exception Handling: Issues and Proposed Notation." Commun. ACM, Vol. 18, No. 12, pp. 683–696.
Goos, G., and J. Hartmanis (eds.). (1983) The Programming Language Ada Reference Manual. American National Standards Institute. ANSI/-MIL-STD-1815-A–1983. Lecture Notes in Computer Science 155. Springer-Verlag, New York.
Gordon, M. (1979) The Denotational Description of Programming Languages, An Introduction. Springer-Verlag, New York.
Graham, P. (1996) ANSI Common LISP. Prentice Hall, Englewood Cliffs, NJ.
Gries, D. (1981) The Science of Programming. Springer-Verlag, New York.
Halstead, R. H., Jr. (1985) "Multilisp: A Language for Concurrent Symbolic Computation." ACM Transactions on Programming Language and Systems, Vol. 7, No. 4, October 1985, pp. 501–538.
Halvorson, M. (2013) Microsoft Visual Basic 2013 Step by Step. Microsoft Press, Redmond, WA.
Hammond, P. (1983) APES: A User Manual. Department of Computing Report 82/9. Imperial College of Science and Technology, London.
Harbison, S. P., III, and G. L. Steele, Jr. (2002) C: A Reference Manual, 5e. Prentice Hall, Upper Saddle River, NJ.
Henderson, P. (1980) Functional Programming: Application and Implementation. Prentice Hall, Englewood Cliffs, NJ.
Hoare, C. A. R. (1969) "An Axiomatic Basis of Computer Programming." Commun. ACM, Vol. 12, No. 10, pp. 576–580.
Hoare, C. A. R. (1972) "Proof of Correctness of Data Representations." Acta Informatica, Vol. 1, pp. 271–281.
Hoare, C. A. R. (1973) "Hints on Programming Language Design." Proceedings ACM SIGACT/SIGPLAN Conference on Principles of Programming Languages. Also published as Technical Report STAN-CS-73-403, Stanford University Computer Science Department.
Hoare, C. A. R. (1974) "Monitors: An Operating System Structuring Concept." Commun. ACM, Vol. 17, No. 10, pp. 549–557.
Hoare, C. A. R. (1978) "Communicating Sequential Processes." Commun. ACM, Vol. 21, No. 8, pp. 666–677.
Hoare, C. A. R. (1981) "The Emperor's Old Clothes." Commun. ACM, Vol. 24, No. 2, pp. 75–83.
Hoare, C. A. R., and N. Wirth. (1973) "An Axiomatic Definition of the Programming Language Pascal." Acta Informatica, Vol. 2, pp. 335–355.
Hogger, C. J. (1984) Introduction to Logic Programming. Academic Press, London.
Hogger, C. J. (1991) Essentials of Logic Programming. Oxford Science Publications, Oxford, England.
Holt, R. C., G. S. Graham, E. D. Lazowska, and M. A. Scott. (1978) Structured Concurrent Programming with Operating Systems Applications. Addison-Wesley, Reading, MA.
Horn, A. (1951) "On Sentences Which Are True of Direct Unions of Algebras." J. Symbolic Logic, Vol. 16, pp. 14–21.
Hudak, P., and J. Fasel. (1992) "A Gentle Introduction to Haskell." ACM SIGPLAN Notices, Vol. 27, No. 5, May 1992, pp. T1–T53.

Hughes, J. (1989) "Why Functional Programming Matters." The Computer Journal, Vol. 32, No. 2, pp. 98–107.
Huskey, H. K., R. Love, and N. Wirth. (1963) "A Syntactic Description of BC NELIAC." Commun. ACM, Vol. 6, No. 7, pp. 367–375.
IBM. (1954) "Preliminary Report, Specifications for the IBM Mathematical FORmula TRANslating System, FORTRAN." IBM Corporation, New York.
IBM. (1956) "Programmer's Reference Manual, The FORTRAN Automatic Coding System for the IBM 704 EDPM." IBM Corporation, New York.
IBM. (1964) The New Programming Language. IBM UK Laboratories, Hursley, England.
Ichbiah, J. D., J. C. Heliard, O. Roubine, J. G. P. Barnes, B. Krieg-Brueckner, and B. A. Wichmann. (1979) "Part B: Rationale for the Design of the Ada Programming Language." ACM SIGPLAN Notices, Vol. 14, No. 6.
IEEE. (1985) "Binary Floating-Point Arithmetic." IEEE Standard 754, IEEE, New York.
INCITS/ISO/IEC. (1997) 1539-1-1997, Information Technology—Programming Languages—FORTRAN, Part 1: Base Language. American National Standards Institute, New York.
Ingerman, P. Z. (1967). "Panini-Backus Form Suggested." Commun. ACM, Vol. 10, No. 3, p. 137.
ISO. (1998) ISO14882-1, ISO/IEC Standard – Information Technology—Programming Language—C++. International Organization for Standardization, Geneva, Switzerland.
ISO. (1999) ISO/IEC 9899:1999, Programming Language C. American National Standards Institute, New York.
ISO/IEC. (1996) 14977:1996, Information Technology—Syntactic Metalanguage—Extended BNF. International Organization for Standardization, Geneva, Switzerland.
ISO/IEC. (2002) 1989:2002, Information Technology—Programming Languages—COBOL. American National Standards Institute, New York.
ISO/IEC. (2010) 1539-1, Information Technology—Programming Languages—Fortran. American National Standards Institute, New York.
ISO/IEC. (2014) 8652/2012(E), Ada 2012 Reference Manual. Springer-Verlag, New York.
Iverson, K. E. (1962) A Programming Language. John Wiley, New York.
Jensen, K., and N. Wirth. (1974) Pascal Users Manual and Report. Springer-Verlag, Berlin.
Johnson, S. C. (1975) "Yacc: Yet Another Compiler-Compiler." Computing Science Report 32. AT&T Bell Laboratories, Murray Hill, NJ.
Jones, N. D. (ed.). (1980) Semantic-Directed Compiler Generation. Lecture Notes in Computer Science, Vol. 94. Springer-Verlag, Heidelberg, FRG.
Kay, A. (1969) The Reactive Engine. PhD Thesis. University of Utah, September.
Kernighan, B. W., and D. M. Ritchie. (1978) The C Programming Language. Prentice Hall, Englewood Cliffs, NJ.
Knuth, D. E. (1965) "On the Translation of Languages from Left to Right." Information & Control, Vol. 8, No. 6, pp. 607–639.
Knuth, D. E. (1967) "The Remaining Trouble Spots in ALGOL 60." Commun. ACM, Vol. 10, No. 10, pp. 611–618.
Knuth, D. E. (1968) "Semantics of Context-Free Languages." Mathematical Systems Theory, Vol. 2, No. 2, pp. 127–146.

Knuth, D. E. (1974) "Structured Programming with GOTO Statements." ACM Computing Surveys, Vol. 6, No. 4, pp. 261–301.
Knuth, D. E. (1981) The Art of Computer Programming, Vol. II, 2e. Addison-Wesley, Reading, MA.
Knuth, D. E., and L. T. Pardo. (1977) "Early Development of Programming Languages." In Encyclopedia of Computer Science and Technology, G. Holzman and A. Kent (eds.). Vol. 7. Dekker, New York, pp. 419–493.
Kowalski, R. A. (1979) Logic for Problem Solving. Artificial Intelligence Series, Vol. 7. Elsevier-North Holland, New York.
Laning, J. H., Jr., and N. Zierler. (1954) "A Program for Translation of Mathematical Equations for Whirlwind I." Engineering memorandum E-364. Instrumentation Laboratory, Massachusetts Institute of Technology, Cambridge, MA.
Ledgard, H. F., and M. Marcotty. (1975) "A Genealogy of Control Structures." Commun. ACM, Vol. 18, No. 11, pp. 629–639.
Lippman, S. B., and J. Lajoie. (2012) C++ Primer, 5e. Addison-Wesley, Upper Saddle River, NJ.
Lischner, R. (2000) Delphi in a Nutshell. O'Reilly Media, Sebastopol, CA.
Liskov, B., R. L. Atkinson, T. Bloom, J. E. B. Moss, C. Scheffert, R. Scheifler, and A. Snyder. (1981) CLU Reference Manual. Springer, New York.
Lomet, D. (1975) "Scheme for Invalidating References to Freed Storage." IBM Journal of Research and Development, Vol. 19, pp. 26–35.
Lutz, M. (2013) Learning Python, 5e. O'Reilly Media, Sebastopol, CA.
MacLaren, M. D. (1977) "Exception Handling in PL/I." ACM SIGPLAN Notices, Vol. 12, No. 3, pp. 101–104.
Marcotty, M., H. F. Ledgard, and G. V. Bochmann. (1976) "A Sampler of Formal Definitions." ACM Computing Surveys, Vol. 8, No. 2, pp. 191–276.
Mather, D. G., and S. V. Waite (eds.). (1971) BASIC, 6e. University Press of New England, Hanover, NH.
McCarthy, J. (1960) "Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I." Commun. ACM, Vol. 3, No. 4, pp. 184–195.
McCarthy, J., P. W. Abrahams, D. J. Edwards, T. P. Hart, and M. Levin. (1965) LISP 1.5 Programmer's Manual, 2e. MIT Press, Cambridge, MA.
McCracken, D. (1970) "Whither APL." Datamation, September 15, pp. 53–57.
Metcalf, M., J. Reid, and M. Cohen. (2004) Fortran 95/2003 Explained, 3e. Oxford University Press, Oxford, England.
Meyer, B. (1990) Introduction to the Theory of Programming Languages. Prentice Hall, Englewood Cliffs, NJ.
Milner, R., R. Harper, and M. Tofle. (1997) The Definition of Standard ML-Revised. MIT Press, Cambridge, MA.
Milos, D., U. Pleban, and G. Loegel. (1984) "Direct Implementation of Compiler Specifications." POPL '84 Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Programming Languages, pp. 196–202.
Mitchell, J. G., W. Maybury, and R. Sweet. (1979) Mesa Language Manual, Version 5.0, CSL-79-3. Xerox Research Center, Palo Alto, CA.
Moss, C. (1994) Prolog++: The Power of Object-Oriented and Logic Programming. Addison-Wesley, Reading, MA.
Moto-oka, T. (1981) "Challenge for Knowledge Information Processing Systems." Proceedings of the International Conference on Fifth Generation Computing Systems. Japan Information Processing Development Center, Tokyo. Republished (1982) by North-Holland Publishing, Amsterdam.

Naur, P. (ed.). (1960) "Report on the Algorithmic Language ALGOL 60." Commun. ACM, Vol. 3, No. 5, pp. 299–314.
Newell, A., and H. A. Simon. (1956) "The Logic Theory Machine—A Complex Information Processing System." IRE Transactions on Information Theory, Vol. IT-2, No. 3, pp. 61–79.
Newell, A., and F. M. Tonge. (1960) "An Introduction to Information Processing Language V." Commun. ACM, Vol. 3, No. 4, pp. 205–211.
Nilsson, N. J. (1971) Problem Solving Methods in Artificial Intelligence. McGraw-Hill, New York.
Pagan, F. G. (1981) Formal Specifications of Programming Languages. Prentice Hall, Englewood Cliffs, NJ.
Papert, S. (1980) MindStorms: Children, Computers and Powerful Ideas. Basic Books, New York.
Perlis, A., and K. Samelson. (1958) "Preliminary Report—International Algebraic Language." Commun. ACM, Vol. 1, No. 12, pp. 8–22.
Peyton Jones, S. L. (1987) The Implementation of Functional Programming Languages. Prentice Hall, Englewood Cliffs, NJ.
Pratt, T. W., and M. V. Zelkowitz. (2001) Programming Languages: Design and Implementation, 4e. Prentice Hall, Englewood Cliffs, NJ.
Remington-Rand. (1952) "UNIVAC Short Code." Unpublished collection of dittoed notes. Preface by A. B. Tonik, dated October 25, 1955 (1 p.); Preface by J. R. Logan, undated but apparently from 1952 (1 p.); Preliminary exposition, 1952? (22 pp., in which pp. 20–22 appear to be a later replacement); Short code supplementary information, topic one (7 pp.); Addenda #1, 2, 3, 4 (9 pp.).
Reppy, J. H. (1999) Concurrent Programming in ML. Cambridge University Press, New York.
Richards, M. (1969) "BCPL: A Tool for Compiler Writing and Systems Programming." Proc. AFIPS SJCC, Vol. 34, pp. 557–566.
Robbins, A. (2005) Unix in a Nutshell, 4e. O'Reilly Media, Sebastopol, CA.
Robinson, J. A. (1965) "A Machine-Oriented Logic Based on the Resolution Principle." Journal of the ACM, Vol. 12, pp. 23–41.
Roussel, P. (1975) "PROLOG: Manual de Reference et D'utilisation." Research Report. Artificial Intelligence Group, University of Aix-Marseille, Luming, France.
Rubin, F. (1987) "'GOTO Statement Considered Harmful' considered harmful" (letter to editor). Commun. ACM, Vol. 30, No. 3, pp. 195–196.
Rutishauser, H. (1967) Description of ALGOL 60. Springer-Verlag, New York.
Sammet, J. E. (1969) Programming Languages: History and Fundamentals. Prentice Hall, Englewood Cliffs, NJ.
Sammet, J. E. (1976) "Roster of Programming Languages for 1974–75." Commun. ACM, Vol. 19, No. 12, pp. 655–669.
Schorr, H., and W. Waite. (1967) "An Efficient Machine Independent Procedure for Garbage Collection in Various List Structures." Commun. ACM, Vol. 10, No. 8, pp. 501–506.
Scott, D. S., and C. Strachey. (1971) "Towards a Mathematical Semantics for Computer Language." In Proceedings, Symposium on Computers and Automation, J. Fox (ed.). Polytechnic Institute of Brooklyn Press, New York, pp. 19–46.
Scott, M. (2009) Programming Language Pragmatics, 3e. Morgan Kaufman, San Francisco, CA.
Sebesta, R. W. (1991) VAX Structured Assembly Language Programming, 2e. Benjamin/Cummings, Redwood City, CA.

Sergot, M. J. (1983) "A Query-the-User Facility for Logic Programming." In Integrated Interactive Computer Systems, P. Degano and E. Sandewall (eds.). North-Holland Publishing, Amsterdam.
Shaw, C. J. (1963) "A Specification of JOVIAL." Commun. ACM, Vol. 6, No. 12, pp. 721–736.
Smith, J. B. (2006) Practical OCaml. Apress, Springer-Verlag, New York.
Sommerville, I. (2010) Software Engineering, 9e. Addison-Wesley, Reading, MA.
Steele, G. L., Jr. (1990) Common LISP The Language, 2e. Digital Press, Burlington, MA.
Stoy, J. E. (1977) Denotational Semantics: The Scott–Strachey Approach to Programming Language Semantics. MIT Press, Cambridge, MA.
Stroustrup, B. (1983) "Adding Classes to C: An Exercise in Language Evolution." Software—Practice and Experience, Vol. 13, pp. 139–161.
Stroustrup, B. (1984) "Data Abstraction in C." AT&T Bell Laboratories Technical Journal, Vol. 63, No. 8, pp. 1701–1732.
Stroustrup, B. (1986) The C++ Programming Language. Addison-Wesley, Reading, MA.
Stroustrup, B. (1988) "What Is Object-Oriented Programming?" IEEE Software, May 1988, pp. 10–20.
Stroustrup, B. (1994) The Design and Evolution of C++. Addison-Wesley, Reading, MA.
Stroustrup, B. (1997) The C++ Programming Language, 3e. Addison-Wesley, Reading, MA.
Sussman, G. J., and G. L. Steele, Jr. (1975) "Scheme: An Interpreter for Extended Lambda Calculus." MIT AI Memo No. 349 (December 1975).
Suzuki, N. (1982) "Analysis of Pointer 'Rotation'." Commun. ACM, Vol. 25, No. 5, pp. 330–335.
Syme, D., A. Granicz, and A. Cisternino. (2010) Expert F# 2.0. Apress, Springer-Verlag, New York.
Tatroe, K., P. MacIntyre, and R. Lerdorf. (2013) Programming PHP, 3e. O'Reilly Media, Sebastopol, CA.
Tanenbaum, A. S. (2005) Structured Computer Organization, 5e. Prentice Hall, Englewood Cliffs, NJ.
Teitelbaum, T., and T. Reps. (1981) "The Cornell Program Synthesizer: A Syntax-Directed Programming Environment." Commun. ACM, Vol. 24, No. 9, pp. 563–573.
Tenenbaum, A. M., Y. Langsam, and M. J. Augenstein. (1990) Data Structures Using C. Prentice Hall, Englewood Cliffs, NJ.
Thomas, D., A. Hunt, and C. Fowler. (2013) Programming Ruby 1.9 & 2.0: The Pragmatic Programmers Guide (The Facets of Ruby). The Pragmatic Bookshelf, Raleigh, NC.
Thompson, S. (1999) Haskell: The Craft of Functional Programming, 2e. Addison-Wesley, Reading, MA.
Turner, D. (1986) "An Overview of Miranda." ACM SIGPLAN Notices, Vol. 21, No. 12, pp. 158–166.
Ullman, J. D. (1998) Elements of ML Programming. ML97 edition. Prentice Hall, Englewood Cliffs, NJ.
van Emden, M. H. (1980) "McDermott on Prolog: A Rejoinder." SIGART Newsletter, No. 72, August, pp. 19–20.
van Wijngaarden, A., B. J. Mailloux, J. E. L. Peck, and C. H. A. Koster. (1969) "Report on the Algorithmic Language ALGOL 68." Numerische Mathematik, Vol. 14, No. 2, pp. 79–218.

Wadler, P. (1998) "Why No One Uses Functional Languages." ACM SIGPLAN Notices, Vol. 33, No. 2, February 1998, pp. 25–30.

Warren, D. H. D., L. M. Pereira, and F. C. N. Pereira. (1979) "User's Guide to DEC System-10 Prolog." Occasional Paper 15. Department of Artificial Intelligence, University of Edinburgh, Scotland.

Watt, D. A. (1979) "An Extended Attribute Grammar for Pascal." ACM SIGPLAN Notices, Vol. 14, No. 2, pp. 60–74.

Wegner, P. (1972) "The Vienna Definition Language." ACM Computing Surveys, Vol. 4, No. 1, pp. 5–63.

Weissman, C. (1967) LISP 1.5 Primer. Dickenson Press, Belmont, CA.

Wexelblat, R. L. (ed.). (1981) History of Programming Languages. Academic Press, New York.

Wheeler, D. J. (1950) "Programme Organization and Initial Orders for the EDSAC." Proc. R. Soc. London, Ser. A, Vol. 202, pp. 573–589.

Wilkes, M. V. (1952) "Pure and Applied Programming." In Proceedings of the ACM National Conference, Vol. 2. Toronto, pp. 121–124.

Wilkes, M. V., D. J. Wheeler, and S. Gill. (1951) The Preparation of Programs for an Electronic Digital Computer, with Special Reference to the EDSAC and the Use of a Library of Subroutines. Addison-Wesley, Reading, MA.

Wilkes, M. V., D. J. Wheeler, and S. Gill. (1957) The Preparation of Programs for an Electronic Digital Computer, 2e. Addison-Wesley, Reading, MA.

Wilson, P. R. (2005) "Uniprocessor Garbage Collection Techniques." Available at http://www.cs.utexas.edu/users/oops/papers.htm#bigsurv.

Wirth, N. (1971) "The Programming Language Pascal." Acta Informatica, Vol. 1, No. 1, pp. 35–63.

Wirth, N. (1973) Systematic Programming: An Introduction. Prentice Hall, Englewood Cliffs, NJ.

Wirth, N. (1975) "On the Design of Programming Languages." Information Processing 74 (Proceedings of IFIP Congress 74). North Holland, Amsterdam, pp. 386–393.

Wirth, N. (1977) "Modula: A Language for Modular Multi-Programming." Software—Practice and Experience, Vol. 7, pp. 3–35.

Wirth, N., and C. A. R. Hoare. (1966) "A Contribution to the Development of ALGOL." Commun. ACM, Vol. 9, No. 6, pp. 413–431.

Zuse, K. (1972) "Der Plankalkül." Manuscript prepared in 1945, published in Berichte der Gesellschaft für Mathematik und Datenverarbeitung, No. 63 (Bonn, 1972); Part 3, 285 pp. English translation of all but pp. 176–196 in No. 106 (Bonn, 1976), pp. 42–244.

Index

T